

Is It a Promise?

How to tell, from the declaration, the body, or the caller — and the full list of things that return one

October 2026

The one question. For every function call you read: *ticket or value*? A Promise is a ticket for a value that arrives later; `await` cashes it. Holding a ticket and treating it as the value is the single most common planted bug.

What makes a function return a Promise (the facts)

Exactly two things. No annotation is required for the Promise to exist.

Fact	Example	Result
The <code>async</code> keyword	<code>async function f() { return 5 }</code>	<code>Promise<number></code> , always, even if the body is instant
The body returns something that is already a Promise	<code>function f() { return fetch(url) }</code>	<code>Promise<Response></code> , passed through

The `: Promise<T>` annotation does not make a function return a Promise; it only documents that one of the two facts holds. It is optional and frequently omitted.

How to tell (the clues, in order of reliability)

#	Clue	Where you look	Example
1	<code>async</code> keyword	the declaration	<code>async function getUser(...), const f = async () =></code>
2	<code>: Promise<T></code> return type	the declaration, after <code>)</code>	<code>function getUser(id): Promise<User></code>
3	the body returns a Promise-maker	the <code>return</code> statements	<code>return fetch(...), return db.query(...)</code>
4	callers <code>await</code> it	every call site	<code>const u = await getUser(id), .then(...), inside Promise.all</code>
5	the name and the job	weakest; a hint only	<code>fetch / load / save / query / read / write / send / connect</code> → usually <code>async</code>

One clue is enough. Clue 1 is certain. Clue 2 is certain when present. Clue 3 needs you to chase the `return`. Clue 4 is the author telling you. Clue 5 only tells you where to look.

Say, when you cannot tell: “Assuming `getUser` is `async`, as the `await` on line 12 suggests; if it is synchronous, line 20’s `await` is harmless but line 31’s missing one is not.” An honest conditional beats a wrong certainty.

Clue 3 in detail: the pass-through

```
function getUser(id: string) {  
  return fetch(`/users/${id}`);  
}
```

No `async`, no annotation, still a Promise. Say: “`getUser` returns whatever `fetch` returns.” Then: “`fetch` returns `Promise<Response>`.” So `getUser` returns `Promise<Response>`. The function is a pipe: whatever goes into the `return` comes out of the call.

return X means the function returns the type of X.

Body	X is	Function returns
<code>return 5</code>	a number	<code>number</code>
<code>return fetch(url)</code>	<code>Promise<Response></code>	<code>Promise<Response></code>
<code>return db.query(sql)</code>	<code>Promise<Row[]></code>	<code>Promise<Row[]></code>
<code>return getUser(id)</code>	whatever <code>getUser</code> returns	the same; chase one more step
<code>return fetch(url).then(r => r.json())</code>	a Promise (<code>.then</code> always gives one)	<code>Promise<any></code>
<code>return new Promise(...)</code>	a Promise	a Promise
<code>return items.map(f)</code>	an array	an array, not a Promise
<code>return await fetch(url)</code>	the unwrapped Response (needs <code>async</code>)	<code>Promise<Response></code> because of <code>async</code>

The chase: follow `return` until you hit an `async` keyword, a known Promise-maker from the list below, or a plain value. Stop there.

Why authors write it: `async function f() { return await fetch(...) }` and `function f() { return fetch(...) }` do the same job; the second is shorter. It is a legitimate style, not a finding.

The one trap in that style:

```
function getUser(id: string) {
  try {
    return fetch(`/users/${id}`); // the Promise leaves the try before it settles
  } catch (e) {
    log(e);                       // never runs for a network failure
  }
}
```

A rejection escapes the `try` because nothing waited inside it. This is the one place `async` + `return await` genuinely matters.

Everything that returns a Promise

Memorize these so the chase can stop when it reaches one.

Language and standard library

Call	Returns	Notes
any function marked <code>async</code>	<code>Promise<whatever it returns></code>	the keyword is the proof
<code>new Promise((resolve, reject) => ...)</code>	<code>Promise<T></code>	the raw constructor
<code>Promise.resolve(v)</code>	<code>Promise<T></code>	a settled ticket
<code>Promise.reject(e)</code>	rejected <code>Promise</code>	
<code>Promise.all([...])</code>	<code>Promise<T[]></code>	rejects on first failure
<code>Promise.allSettled([...])</code>	<code>Promise<{status, value reason}[]></code>	never rejects
<code>Promise.race([...])</code>	<code>Promise<T></code>	first to settle
<code>Promise.any([...])</code>	<code>Promise<T></code>	first to fulfil
<code>p.then(f)</code>	<code>Promise<result of f></code>	chaining always yields a Promise
<code>p.catch(f)</code>	<code>Promise</code>	same
<code>p.finally(f)</code>	<code>Promise</code>	same
<code>fetch(url, opts?)</code>	<code>Promise<Response></code>	resolves on ANY HTTP status; rejects only on network failure
<code>res.json()</code>	<code>Promise<any></code>	the body, parsed
<code>res.text()</code>	<code>Promise<string></code>	
<code>res.blob()</code> , <code>res.arrayBuffer()</code> , <code>res.formData()</code>	<code>Promise<...></code>	
<code>import("./m")</code>	<code>Promise<Module></code>	dynamic import
<code>crypto.subtle.*</code>	<code>Promise</code>	Web Crypto
<code>navigator.clipboard.readText()</code> / <code>writeText()</code>	<code>Promise</code>	browser
<code>structuredClone</code>	a value, NOT a Promise	listed to prevent the mistake
<code>JSON.parse</code> , <code>JSON.stringify</code>	a value, NOT a Promise	synchronous; throw/return at once
<code>setTimeout(fn, ms)</code>	a timer id, NOT a Promise	becomes one only when wrapped: <code>new Promise(r => setTimeout(r, ms))</code>

Node built-ins

Call	Returns	Notes
<code>fs.promises.readFile</code> , <code>writeFile</code> , <code>readdir</code> , <code>stat</code> , <code>mkdir</code> , <code>unlink</code> , <code>access</code>	<code>Promise</code>	<code>import { readFile } from "fs/promises"</code>
<code>fs.readFileSync</code> and every <code>*Sync</code>	a value	synchronous; blocks
<code>fs.readFile(path, cb)</code>	<code>undefined</code>	callback style, not a Promise; wrap with <code>util.promisify</code>

Call	Returns	Notes
<code>util.promisify(fn)</code>	a function that returns a Promise	
<code>stream/promises</code> <code>pipeline</code> , <code>finished</code>	Promise	
<code>timers/promises</code> <code>setTimeout(ms)</code>	Promise	the promisified sleep
<code>child_process</code> <code>exec</code> via <code>promisify</code>	Promise	plain <code>exec</code> is callback style
<code>dns.promises.*</code>	Promise	
<code>events.once(emitter, "name")</code>	Promise	
<code>http.request</code>	a request object, NOT a Promise	event-based

Database and HTTP clients (every one of these)

Library	Returns a Promise from
<code>pg</code>	<code>client.query(...)</code> , <code>pool.query(...)</code> , <code>pool.connect()</code>
<code>mysql2/promise</code>	<code>conn.query(...)</code> , <code>conn.execute(...)</code>
<code>prisma</code>	<code>prisma.user.findMany()</code> , <code>.findUnique()</code> , <code>.create()</code> , <code>.update()</code> , <code>.delete()</code> , <code>\$transaction</code>
<code>mongoose</code> / <code>mongodb</code>	<code>Model.find()</code> , <code>.findOne()</code> , <code>.save()</code> , <code>collection.insertOne()</code> , <code>.updateOne()</code>
<code>knex</code>	every query builder chain once awaited; <code>.then</code> -able
<code>typeorm</code> / <code>sequelize</code> / <code>drizzle</code>	repository and model methods
<code>ioredis</code> / <code>redis</code>	<code>client.get()</code> , <code>.set()</code> , <code>.del()</code> , <code>.incr()</code> , <code>.expire()</code>
<code>axios</code>	<code>axios.get()</code> , <code>.post()</code> , <code>axios(config)</code> → <code>Promise<AxiosResponse></code>
<code>node-fetch</code> , <code>undici</code> , <code>got</code> , <code>ky</code>	same shape as <code>fetch</code>
<code>@aws-sdk/*</code> v3	<code>client.send(command)</code>
<code>firebase</code> / <code>firestore</code>	<code>getDoc()</code> , <code>setDoc()</code> , <code>addDoc()</code> , <code>getDocs()</code>
<code>stripe</code> , <code>twilio</code> , <code>sendgrid</code> , any SDK	nearly every method

Frameworks

Context	Promise?
Express <code>app.get("/", async (req, res) => ...)</code>	the handler returns a Promise; Express 4 ignores it, so errors thrown inside are NOT caught without <code>next(err)</code> or a wrapper
Express <code>res.json()</code> , <code>res.send()</code> , <code>res.status()</code>	NOT Promises; synchronous, chainable; and they do not stop the function
Next.js <code>getServerSideProps</code> , route handlers, server actions	<code>async</code> ; return Promises
Jest/Vitest <code>it("...", async () => ...)</code>	the test runner awaits it
React <code>useEffect(async () => ...)</code>	× not allowed; <code>useEffect</code> expects a sync function or a cleanup; wrap an async call inside
Event handlers <code>onClick={async () => ...}</code>	allowed; the Promise is ignored, so add <code>.catch</code>

Your own code

Shape	Promise?
<code>async function f() {}</code>	yes
<code>const f = async () => {}</code>	yes
<code>class A { async m() {} }</code>	yes
<code>function f() { return fetch(...) }</code>	yes, pass-through
<code>function f() { return someAsyncFn() }</code>	yes, pass-through
<code>function f() { someAsyncFn() }</code> with no <code>return</code>	NO: returns <code>undefined</code> ; the Promise is dropped (fire-and-forget bug)
<code>function f() { return [1, 2].map(x => x) }</code>	no
<code>function f(): Promise<void> { ... }</code> without <code>async</code>	must return a Promise in the body or it will not compile

Things that look async and are not

Looks like	Actually
<code>JSON.parse(s)</code>	synchronous; throws immediately on bad input
<code>arr.map(async x => ...)</code>	<code>map</code> itself is synchronous and returns <code>Promise[]</code> ; the Promises are inside the array
<code>arr.forEach(async x => ...)</code>	<code>forEach</code> is synchronous and returns <code>undefined</code> ; the Promises are thrown away
<code>setTimeout(fn, 0)</code>	schedules; returns an id; not awaitable
<code>localStorage.getItem</code>	synchronous
<code>fs.readFileSync</code>	synchronous
<code>res.json(data)</code> in Express	synchronous send; nothing to await
<code>console.log</code>	synchronous
<code>Date.now()</code>	synchronous
<code>crypto.randomUUID()</code>	synchronous (<code>crypto.subtle.*</code> is async)

Decision procedure for any call `g(...)`

1. Is `g` declared `async`? → Promise. Stop.
2. Does its declaration say `: Promise<...>?` → Promise. Stop.
3. Is `g` on the Promise-makers list above (`fetch`, `db.query`, `.then`, `new Promise`, `fs/promises`, any client SDK)? → Promise. Stop.
4. Open `g`'s body. Find every `return`. Is any returned expression a call that passes steps 1–3? → Promise. Stop.
5. Is `g` awaited or `.then`'d anywhere else in the file? → the author thinks it is a Promise; treat it as one and verify with 1–4.
6. None of the above → a plain value.

Then, for every call site of `g`: if it is a Promise, is it `awaited`, `returned`, `.then`'d, or inside `Promise.all`? If not, that is the finding.

What the type says once you know

Written	What you hold after <code>await</code>
<code>Promise<number></code>	a number
<code>Promise<User></code>	a User object
<code>Promise<User[]></code>	an array of Users
<code>Promise<User null></code>	a User or null; check before <code>.name</code>
<code>Promise<void></code>	nothing useful; the point was the side effect
<code>Promise<Response></code>	a fetch Response; still need <code>res.ok</code> and <code>await res.json()</code>
<code>Promise<any></code>	anything; checking is off (what <code>res.json()</code> gives you)
<code>Promise<unknown></code>	anything; must narrow before use
<code>Promise<User>[]</code>	not awaitable directly; <code>await Promise.all(...)</code> first, then <code>User[]</code>
<code>Promise<Promise<User>></code>	does not exist; Promises flatten; it is just <code>Promise<User></code>